LANCE

Where the project stands

State report as of 2 November 2026

LANCE project

Written by specialist strands, independently reviewed and synthesised

Last revision: 2026-11-02 17:00 (Europe/Brussels, UTC+01:00)

Documentation state as of 2 November 2026; no new experiment is reported

Abstract

LANCE is a platform for the security audit of authorised connected-object networks, developed in the context of the UMONS work of the NATO SPS G6259 programme. It asks one measurable question : *when does a language model find something that a competent automation cannot find, given the same tools, the same evidence and the same budgets ?*

This report answers a reading question : where does the project stand today ? Its first pages give the answer as a whole — what the system does, where the model actually decides, which three policies are compared, and what blocks the conclusion (Section 1). The parts that follow are the evidence behind that answer : the pipeline architecture, the audit protocols and their evidence contracts, the experimentation harnesses and their sandbox, the execution infrastructure and the delivery pipeline, and the benchmark itself.

The position is unchanged since the previous report of 30 October 2026 : the system is built, instrumented and documented, and its chain of evidence is sound. The scientific question remains open. No superiority of the model is established, no confidence interval can be computed, and the obstacle is no longer the computation but the absence of independent families to compute on.

The document consistently distinguishes what is implemented, what is measured and what remains proposed. No new experiment is reported : the measurements cited are those already recorded, with their date. The limits of each component are explicit, including those of the sandbox, whose guard is not an exhaustive native-code sandbox.

Contents

1 Where the project stands **2**

2 Vocabulary **6**

3 Scope and method **7**

4 How an audit is built **7**

5 Audit protocols and evidence contracts **10**

6 Harnesses, and a sandbox that measures nothing **14**

7 Execution infrastructure and delivery pipeline **16**

8 The benchmark, and the distance left to a result **19**

References **23**

9 Conclusion **24**

1 Where the project stands

This section answers the question of the report before the evidence arrives. It holds in four pages : what an audit does and who decides at each step (Figure 1), which three policies are compared and what is missing for the comparison to be fair (Figure 2), what blocks the conclusion and how the project got there (Figure 3 and Table 1). The five parts that follow are the proof.

1.1 The project in one paragraph

A security auditor takes a network it has never seen, finds what is wrong with it, proves each finding rather than guessing it, and writes a report an operator can act on. LANCE does this with a language model in the loop. The programme asks a precise question : *the model earns its place only where a competent script would fail — does that situation exist, and can it be measured ?*

The answer requires three things, and the report follows that order. A system whose behaviour is traceable, so that a result can be attributed to a cause (Figure 1). A comparison in which the only thing that changes is the reasoning capacity, not the information or the tools (Figure 2). Scenarios where the two candidates genuinely differ (Figure 3). The first exists, the second is half built, the third does not yet exist. That is the whole state of the project.

1.2 What an audit does, and where the model decides

An audit runs in six phases. Each phase produces one document that the next one reads, and nothing else crosses between them. Figure 1 is the map to keep in mind for the rest of the report : it shows the six phases, what each produces, and — the point that matters — *who* decides at each step.



Figure 1 – An audit in six phases, and where the model's freedom actually sits. The model decides at phases 2, 3, 4 and 5 : which probes to run, which weaknesses to hypothesise, how to test each one, and how far to push an intrusion. What the code decides is what becomes verifiable : phase 1 projects its evidence file from the graph tools and refuses a submission that misses a device. When the model's deliverable at phase 5 is missing or invalid, the code writes the assessment from the tool ledger, and a completed phase never certifies that the objectives were reached. Phase 6 assembles the report from section sheets, with an agent that declares no tool and runs a single turn. The score is produced elsewhere, by an offline evaluator that recounts the log against an oracle fixed before the run.

The log is the source, not the model's conclusion. Every tool call is archived with a reference, a refusal or an exception included. A finding that cites a piece of evidence which does not appear in the log is rejected. This is what makes a result attributable : the report states what happened, not what the policy believed happened.

The oracle precedes the run. The expected answer is fixed before the policy runs, in a file the policy never receives. A finding that contradicts the oracle invalidates the trial instead of correcting it. This precedence is what prevents the answer from being adjusted after the fact.

The score is recomputed, never taken on trust. The SSH, MQTT, API and V0 evaluators receive only the briefing, the oracle, the journal and the final decision, and they refuse a decision that cites an observation absent from the journal. The benchmark score of a pipeline run does not perform that check ; the confirmed metrics it would need are not emitted by the harnesses today.

Where this lives. Phase registry `src/agent/phases/registry.py` ; governance boundary `src/agent/core/executor.py` ; evaluators under `src/benchmark/` and `benchmarks/experiments/`.

1.3 The three policies compared on the same scenario

The programme compares three ways of answering the same question on the same scenario. They receive the same targets and are judged by the same evaluator ; what differs is how the answer is produced. The platform also offers four execution modes — `lance` (the full pipeline, with a model), `rules` (analysis and verification only — scanner and aggregation, with no model at all), `vanilla` (a model exploring an address range) and `scripted` (versioned procedures) ; only `lance` is scored against ground truth ; the others are not.

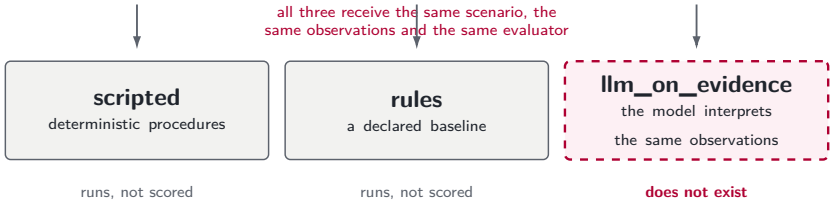


Figure 2 – The three policies and what is missing. The third, `llm_on_evidence`, would give the model the observations the rules already receive and nothing else. It does not exist, and without it the comparison measures verification rather than interpretation.

Why the third policy is the one that matters. An exploration where the model roams a network freely (`vanilla`) changes too much at once : the targets chosen, the order of the probes and the interpretation all differ, so a better score cannot be attributed to reasoning. A policy that receives the scanner's observations and must produce findings backed by a citation of those observations isolates exactly one variable : *who interprets*. That is the question the programme asked.

What currently favours the rules. Four asymmetries separate the declared baseline from the model. The rules arm executes a deterministic probe plan for each finding, while the model must find its own sequence. In the `rules` mode the analysis phase runs without a model at all, so no hypothesis is lost in translation. The scanner's findings feed both aggregations, but there they are the whole of the rules arm rather than a supplement to it. And on the compact path the pipeline writes the scanner's findings over the model's per-device sheet, an asymmetry that runs the other way. Until these are removed or mirrored, an equal score would say nothing.

1.4 What blocks the conclusion

The obstacle is not the measurement machinery, which is now sound, but the material it would need. Figure 3 shows the chain as it stands and the one observation that changed everything on 30 October 2026.

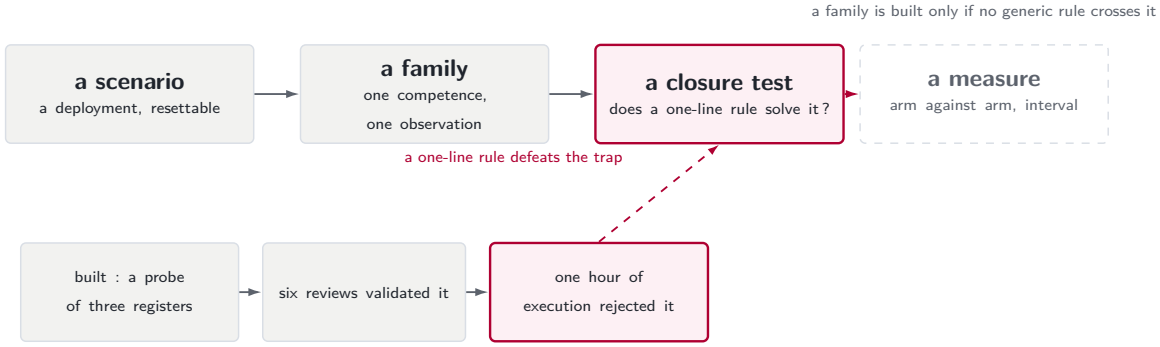


Figure 3 – The chain from a scenario to a measurement, and where it breaks. Every stage exists except the measure. The episode of 30 October shows why the closure test moved ahead of the reviews : one hour of observation rejected a design that six readings had validated.

What the closure test is. Before building a family, one asks whether a rule that solves the case can be written in a single line. If it can, the family measures the omission of a declaration rather than a limit of interpretation, and it is not built. The test is cheap ; running it before writing eight fixtures is the change of method this month produced.

Why this is a blockage and not a delay. The inference tool was repaired on 30 October : it now computes a one-sided p-value against the pre-registered threshold, refuses to emit a figure below three families, and requires five to decide. It is therefore correct, and idle : there is nothing to compute on. Building more families of the same kind would not change the arithmetic.

1.5 What must be decided

The blockage is not technical, so the way out is not more machine time. Three choices govern what follows, and none of them is taken yet.

1. Families. Build families that a one-line rule does not cross, applying the filter that one hour of observation established, or publish a bounded methodological result and declare it as such. The filter matters more than the count : a family crossed by a generic rule measures nothing, and five of them measure nothing either.

2. The opening condition. Write the competent reference automation that D1+ names, or renounce explicitly the hypotheses that depend on it. Several families remain undecidable until this is settled.

3. The comparable arm. Give the model the observations the rules already receive, or state that the comparison measures verification rather than interpretation. The second option is legitimate and much cheaper ; it is not the question the programme asked.

These three decisions together determine whether an inference can one day exist.

1.6 The state in one table

Table 1 – The state of the project on 2 November 2026, and where the proof is.

Question	Answer	**Proof**
Is an audit traceable ?	Yes. Every tool call is archived with a reference, bounded, and a piece of evidence that cannot be archived stops the run instead of leaving a silent gap. The guards are application-level.	**Part 4**
Does the harness run outside the laboratory ?	Yes, without broker and without a model, under a guard whose blind spots are declared. It instruments a contrast ; it measures nothing.	**Part 6**
Are the protocols' contracts sound ?	Yes. Briefing, private oracle and offline evaluator are separated, and the oracle is fixed before the run.	**Part 5**
Can results be delivered reproducibly ?	Yes. Push, validate, deploy to one of three instances ; no execution on a workstation. Single points of failure declared.	**Part 7**
Does the model beat the rules ?	**Unknown, and not yet measurable.**	**Part 8**
How much of the corpus needs interpretation ?	Two counts from two artefacts, never to be cited one for the other. Binary index : 282 entries by signature, 6 by logic (2.1 %). Interpretive audit : 243 decidable by a well-designed rule, 35 mixed, 4 chaining, 6 logic (84.4 %) — a sum over categories, not a field per entry.	**Part 8**
Is the measurement statistically valid ?	The tool is repaired and refuses an indefensible figure ; the families that would give it data do not exist.	**Part 8**

1.7 How far the project goes

The state of each item — reached or not — is in the project's current state page (`objectifs/ETAT-CURANT.md`), which this checkpoint does not repeat. What that table cannot carry is the other half : *what would end each blockage*, as a criterion one can observe rather than a hope.

Table 2 – Each blockage, what would cross it, and how one would know. No term here needs the glossary : the table is meant to be read before it.

Blockage	What would cross it	How one would know
No situation a rule cannot decide	a case where no one-line rule settles the answer	the closure script runs and refuses to admit it
No comparable arm	the model interpreting the very observations the rules receive	the same scenario scored on both, by the same evaluator
The competent reference is a doctrine	a written reference automation standing in for it	it passes a family that the model also passes
Corpus built for rules	scenarios where a well-written rule can only abstain	the interpretive share rises, recomputed entry by entry

This is where the report stops. Everything up to a verified measurement is built ; the thing to be measured is not. The next result the project can legitimately produce is *a family that survives its closure test*, not a better score — and it is the only line of the table whose proof needs no new agreement on method.

2 Vocabulary

The report uses a small vocabulary throughout. Each term is defined here, in the order a newcomer meets them, so that no later section has to introduce it.

2.1 What is compared

Policy (politique)
: One way of carrying out an audit. The programme compares three : a declared automation, a second automation that follows convention, and an agent driven by a language model.

Arm (bras)
: One concrete implementation of a task inside a family. The arms share the briefing and the journal ; only the interpreter changes. A family is typically run with three : the declared rules, a script reading by convention and the model. The witness is a value inside the fixture, not an arm.

D1
: The declared automation : the rules written explicitly for the task.

D1+
: D1 *augmented with every rule derivable from the public inputs.* This is the reference a model must beat for a result to mean anything. If a competent reader who knows the briefing can state the rule, D1+ contains it, and the family measures nothing.

Family (famille)
: A group of variants testing the same competence. Its variants count as a single observation, because they test one thing. Two families share no simulator, no vocabulary and no failure mechanism.

Closure test (test de clôture)
: The check performed before measuring a family : a competent automation must fail. If it succeeds, the family measures nothing about the model and is rejected. The test must be executable code, not a doctrine.

Filter (filtre préalable)
: The cheaper check that precedes construction : if the rule solving the case can be written in one line, a script will have it.

2.2 What makes a result verifiable

A result is verifiable when it can be recounted by someone who does not trust the system. The report uses five objects for that, each answering one question.

Briefing (public)
: Everything given to an arm before it runs : the task, the target, the tools. It never contains the posture or the expected verdict.

Oracle (private)
: The expected answer, written before the run and never shown to the arms. It states which state was installed and which fact is observable, never which decision the policy should return.

Witness (témoin)
: A fresh value drawn when the fixture starts, whose value appears nowhere in the briefing. It proves that what was observed is fresh and not a stale artefact.

Journal (journal)
: The archived log of every tool call made during a run, with its arguments, result and reference. It is what the evaluator recounts ; the policy's own conclusion is not trusted.

Evaluator (offline)
: The program that rereads the journal and recomputes the outcome. A cited piece of evidence absent from the journal is rejected.

A **fixture** is the state installed on the target before a trial : a healthy posture, a vulnerable one, or a frozen corpus read offline. A **policy** is one way of carrying out an audit (a declared automation, an agent driven by a model). An **arm** is one concrete implementation of a task inside a family ; arms share the briefing and the journal, and only the interpreter changes.

2.3 How results are judged

Funnel
: The three stages of a finding, in order : `candidates`, `filtered`, `confirmed`. Only the last counts as a true positive.

Strict-v3
: The normalised vulnerability contract and the metrics derived from it. The `confirmed.*` metrics count only findings whose evidence has been independently validated against the tool log.

Campaign admission
: The set of conditions a campaign must meet before its results can be aggregated. When one is unsatisfied, the results remain descriptive and are declared as such.

V0
: The validity gate placed before every hypothesis : a control task that is solvable without inference. If either arm fails it, no downstream result is readable.

H1 to H6
: The six hypotheses of the register, each naming a kind of competence a model might have where a script does not. H2 is the only one measured ; H4 is the one the rejected family attempted.

C2
: A candidate scenario rejected after two adversarial reviews, on paper, before any run. Its rejection is a published negative result.

Posture
: The state of a fixture with respect to security : healthy (no flaw installed) or vulnerable (flaw installed).

3 Scope and method

This report answers a reading question : *how is the system built, and where does it actually stand ?* It does not replace the reference pages of `docs/`, which describe the current behaviour, nor the detailed protocols of `research/`, which carry the reasoning. It succeeds the report of 30 October 2026 and does not revise it : that folder stays as the archive of its state.

The body was written by specialist strands working on the `dev/1` branch, then reviewed against that same code by an independent reader. This revision changes the order and the register : the state comes first and the evidence after, and the engineering detail was moved to the margins so that the argument stays readable. No figure of this report is new ; all of them are recomputed from the repository or reported as read in a document.

The report is dated : it describes a state, not a contract. Later behaviour may contradict it without making it false. Statements that cannot be verified in the repository are flagged as such rather than filled in.

4 How an audit is built

In brief. An audit goes through six phases, each writing one document that the next one reads. The application is the thinnest part : it accepts a request, allows one run per instance, streams events and reads the artefacts back. The engine is a pipeline that wraps every tool call in a governance boundary : a call is checked against the budgets, archived with a reference, and

archived even when it is refused or fails. The policy only ever reasons over evidence that the tools produced, and the score comes from a separate evaluator, not from the policy.

What the code enforces, and what it does not. What is enforced : a lost piece of evidence stops the run ; a deliverable that misses a device is rejected ; the report is assembled deterministically ; the phase-five status can never be certified. What is not guaranteed : the budgets only exist when they are passed explicitly, the laboratory lock is optional, and two scenarios on a shared laboratory are not mutually excluded.

4.1 The application does not carry the logic

The web layer validates the request, guarantees a single active run per instance, streams the events and re-reads the artefacts. The strategy is chosen by `make\_runner`, not by the interface : for `lance` and `rules` it instantiates a pipeline and wraps it in an adapter that reuses the same object and its public API ; for `vanilla` and `scripted` it builds a standalone runner that shares with the pipeline only the tool boundary, the evidence log, the cost tracking and the laboratory reservation.

Table 3 – Execution modes and their boundary.

Mode	Engine	Model	Effects
`lance`	full pipeline, six phases	required	scenario, deployment, evaluation
`rules`	pipeline, phases 3–4	none	scenario and deterministic audit
`vanilla`	standalone runner	required	bounded network probe on an address range
`scripted`	standalone runner	none	versioned procedures

The `rules` mode is not a second engine : the factory sets the decision policy, restricts the phases to analysis and verification, and refuses any provider, any direct network access and any monetary budget. The two standalone modes, in turn, refuse a scenario and every harness-specific setting. The qualifier `full/compact` designates the orchestration, never the type of runner.

4.2 The six phases and what each hands over

The registry assigns to each phase a prompt, a tool group, a validator and a deliverable ; the routing is an explicit table, not a dynamic import.

Table 4 – Phases, main deliverables and associated artefacts.

Phase	Main deliverable	Who decides	Associated artefacts
1	`01\_graph\_analysis.md`	code (projection of the graph tools)	`01\_graph\_evidence.json`
2	`02\_recon.md`	the model chooses the probes	`02\_recon\_evidence.json`, `02\_topology\_edges.json`
3	`03\_vuln\_analysis.json`	the model writes the hypotheses	`03\_vuln\_analysis\_raw.json`, `03\_phase3\_status.json`, per-device sheets
4	`04\_exploitation.json`	the model tests one flaw per micro-agent	`execution\_errors` diagnostics
5	`05\_intrusion.json`	code (assessment rendered from the log)	`05\_intrusion\_context.json`
6	`06\_report.md`	code (assembled from section sheets)	`06\_report\_analysis.md`

Phase 1 projects the results of the graph tools into a log of facts independent of the model's discourse, and the runner rejects a submission whose per-device coverage is incomplete. Phase 2

lets the model choose the probes, but the recon document is rendered from the tool log rather than copied by the model. Phase 3 runs in three steps — deterministic scanning, one analyst per device, then an aggregation that normalises, deduplicates and projects the findings — the middle step being skipped in the `rules` mode, which has no model at all. Phase 4 launches one micro-agent per canonical finding. Phase 5 is an agent with its own tools and a turn budget ; when its deliverable is missing or invalid, the code writes the assessment from the ledger, and the objectives are never certified : the file records `incomplete` or `blocked`. Phase 6 is the extreme case : its agent declares no tool and runs a single turn, so that the report is assembled and the model writes only a bounded analyst note.

4.3 Graph and run context

The graph is built in two layers : a loader reads the infrastructure description into data objects, then populates a backend. The backend is an abstract interface with a network-graph implementation, so that another engine can be substituted without touching the tools. Two details matter for reading results : the neighbour query is the sorted union of successors and predecessors, and path search uses an undirected view so that a path is found in both directions.

The state belongs to the run, not to the process. Each pipeline creates an immutable run context carrying the backend, the infrastructure, the vulnerability reports, the risk scores, the scenario or discovery topology and the weighted graph. The graph tools retrieve that context without changing the arguments exposed to the model ; thread workers capture the owner explicitly rather than relying on implicit propagation.

4.4 The tool governance boundary

All profiles and all modes go through the same wrapper. On each call it checks the previous piece of evidence, the stop request and the budgets ; increments a counter and refuses the call if the ceiling is reached ; assigns a reference ; applies the scope refusals ; executes ; and archives an entry, including for a refusal or an exception.

Table 5 – Controls applied to every tool call and the consequence of a failure.

Control	Mechanism	Consequence of a failure
Evidence integrity	a flag set when the archive cannot be written	the run stops, and is not verifiable
User stop	a stop event, in the experimental scope	the run stops
Time budget	monotonic clock against the limit	the run stops
Monetary budget	the cost tracker, on observed usage	the run stops
Call budget	a counter taken under the archive lock	the run stops
Scope	the destination fields against the run's target network	the call is refused and archived
Free execution	the execution tool	refused as unverifiable scope

Two properties matter for interpreting a result. A failure to write the log is never ignored : it interrupts the run rather than leaving untraced actions, which is the difference between an audit and a log that only records what went well. And the cost limit applies to observed usage, not to a reservation of calls still in flight, so a run can exceed its monetary budget by the cost of the calls it had already started.

4.5 Artefacts, cost tracking and retention

The run folder is created exclusively under the output tree ; in the event of a collision a hexadecimal suffix is appended and no earlier folder is overwritten. The run metadata records the commit, the policies and the limits, and the cost summary is written in the finalisation block, hence even when the run raises. A submission bundle is a zip equipped with a SHA-256 manifest that is re-checked after writing ; the download route excludes private files and hidden entries.

The price of a phase is resolved once and recorded with its source, so that a completed run cannot be revalued after the fact ; the summary distinguishes the cumulative duration of the agents from the wall-clock time, which are not the same measure.

4.6 What the architecture does not guarantee

The isolation is per run for the deliverables and the graph, but not entirely so : the phase services may still call the common engine, and the simultaneity of two scenarios on a shared laboratory is not guaranteed. The laboratory reservation is active only when its variable is set, and is reentrant only within one thread. Without explicit parameters, the interactive pipeline imposes neither a call ceiling nor an analysis timeout per device, so the budgets of table 5 are conditional rather than structural.

Two points could not be established by reading the code and are therefore not asserted. No archiving or compaction mechanism for run folders was found — the search covered `tarfile`, `shutil.make\_archive` and equivalents — so retention appears to be that of the file system. And the exact field content of the provider diagnostics log was not verified line by line. The recon evidence file is produced by its own phase and read by the scanner, the compact validator and the report renderer. Finally, a completed status means that the runner finished its work, never that the target was exhaustively covered nor that an access was proven.

Where this lives.

— `docs/architecture/run-modes.md` for the modes ; `src/agent/run\_modes.py` for the selection.
— Phase modules : `src/agent/phases/registry.py`, `PHASE\_MODULES`. Phase-1 projection : `GraphPhase.\_build\_graph\_evidence\_projection`.
— `src/loader.py` and `src/graph\_backend.py` ; context in `src/agent/core/graph\_context.py`.
— Implementation : `wrap\_tool` and `check\_execution\_limits`, `src/agent/core/executor.py`.
— Collection contract : `src/benchmark/artifacts.py`. Pricing : `src/agent/cost\_tracker.py`.

5 Audit protocols and evidence contracts

In brief. Each family splits what it tests into three artefacts that the code keeps apart. The *briefing* states only the task, never the expected answer. The *private oracle* states which state was installed and which fact is observable, and is written before the policy runs. The *offline evaluator* re-reads the log and the final decision, and recomputes the outcome. A success is therefore never a declaration ; it is a fact that can be recounted from a journal the policy did not write.

What this buys, and what it does not. It buys freshness and attribution : a witness drawn at each trial proves the value was actually read, and an observed disclosure that contradicts a protected oracle invalidates the trial instead of being counted as a success. It does not buy generalisation : each family covers one protocol, and the V0 control of fidelity covers MQTT only.

5.1 Implemented families

The repository contains five families, each with a centralised contract file, a deterministic arm, a bounded LLM arm and a launcher reserved for the laboratory.

Table 6 – Families, what they measure and what they leave out of scope.

Family	Contract	What it measures	What it does not measure
SSH	`ssh\_reference.py`	Whether a dictionary credential reads the witness object that the legitimate control also reads	Fixture reset, provenance of the model
MQTT	`mqtt\_reference.py`	Whether an anonymous subscriber receives the fresh private witness ; in a healthy posture, the evaluated refusal	Purge of retained messages, broker reset
API	`api\_reference.py`	Coverage of the four authorisation boundaries	The baseline limit is *declared* : the script reads no collection
V0	`v0\_control.py`	Fidelity of execution : a public procedure executed and proved	An assumption about the model
H2	`mqtt\_formats.py`	Reading of a frozen MQTT presentation	Network collection, field semantics, statistical independence

The limit is not merely editorial : it is recorded in each result, under the key `scope`, and in `measurement\_status:evidence\_only\_not\_admitted`.

5.2 A three-part contract

Each family separates three artefacts, and the separation is enforced by the code.

The public briefing contains only the task. `ssh\_reference.validate\_briefing` requires exactly the expected set of fields, which forbids adding a posture or a verdict. The SSH dictionary cannot contain the control identity, and `mqtt\_reference.validate\_briefing` refuses a public subject equal to the private subject. The posture is never written in the briefing : it is inferred from the oracle.

The private oracle is bound to the briefing by the latter's digest, and its fields are bounded by a closed set. It states which state was installed (F0 healthy or F1 vulnerable) and which fact is observable ; it never states which decision the policy should return.

The offline evaluator rereads the log and the terminal decision, then recomputes the outcome. The evidence file is `tool\_calls.jsonl` for SSH, MQTT, API and V0 ; it is `journal.jsonl` for H2, reattached by `Journal.replayed`, which refuses a record originating from another fixture. In a pipeline run each record carries a reference produced by the shared executor ; the sandbox writes its own journal instead.

5.3 The oracle : construction and guarantees

The oracles are produced by `oracle\_for` functions, present in three modules. SSH builds its own inline, from `case\_configuration`, which draws the object with `secrets.token\_hex(32)` ; MQTT produces two fresh witnesses per trial ; the API randomises organisations, users, objects and collections at each trial.

Three barriers prevent deriving the answer from the policy. The first is structural : the evaluator receives only the briefing, the oracle, the journal and the decision, and the oracle is written before the arm runs. The second is cross-validation : a briefing that would name an oracle file fails, and an unbound oracle fails. The third is the contradiction verified over the entire journal : under a protected oracle, an observed disclosure makes the trial `oracle\_conflict` rather than a success. The oracle therefore cannot be adjusted after the fact.

The witness (`canary` in the code) is a fresh value drawn at each fixture start, whose digest is in the oracle and whose value appears nowhere in the briefing. It proves freshness and qualifies the control : the legitimate control must have read the value. Without a witness, an observed refusal might be mere silence, which MQTT explicitly refuses.

5.4 The offline evaluator : recount and statuses

The terminal decision is never taken on trust alone. The evaluator requires that its evidence references be a subset of the journal, and otherwise returns `outcome:unknown`. It then recomputes per observation type : an archived request must be identical to the requested request, and the payload is extracted only if the return code and the framing are consistent. Unbound evidence does not count, even when cited.

In F1, success requires a finding, a proven weak access and a functional control. In F0, it requires the absence of a finding, a functional control and the explicit refusal of each pair of the dictionary. The code distinguishes *non-success* (trial carried to completion, contract not satisfied) from *missing* (evidence or decision absent). The distinction is not cosmetic : a missing item does not enter a success-rate denominator, a non-success fills it. In V0, `instrument\_defect` is added when the script prevails and the LLM fails : this is a defect of the instrument, not a weakness of the model.

5.5 The V0 control

V0 is the validity gate, not a hypothesis. The task is solvable without inference : every subject, identifier and window appears in the briefing.

A *saturating control* is a control that both arms pass everywhere : it detects no defect. The manifest `benchmarks/experiments/v0-control-manifest.json` freezes `X=0.9`, 24 steps and 3 repetitions, and declares `saturating:true` with its reading : the control cannot detect a defect rarer than one failure in three. The evidence is a run with three repetitions in which both arms succeed everywhere. The threshold is read by `frozen\_threshold`, which refuses a manifest that is not frozen or that was calibrated on a different number of steps.

5.6 H2 : three arms, one absorbed prediction

H2 tests whether a model reads a presentation that no declared parser covers. The eight fixtures of the corpus cross `\{reference,variant\}` and `\{raw,normalized\}`, two fixtures per group. Three arms share the same journal and the same verdict vocabulary.

Table 7 – Success rate per group for the three H2 arms (measurement of 22 October).

Group	Declared parser	By convention	LLM
`reference/raw`	1.0	1.0	1.0
`reference/normalized`	1.0	1.0	1.0
`variant/raw`	0.0	**1.0**	**0.5**
`variant/normalized`	1.0	1.0	1.0

The LLM gain holds for one fixture in two. But the conventional arm covers both without a model, since the convention suffices to read the variant. The control therefore publishes `convention\_gain\_absorbed:true` : what the LLM gains over the declared parser, a script that tries also gains. The prediction remains literally upheld, but it no longer distinguishes the model from the declaration.

5.7 The strict-v3 contract and confirmed metrics

The contract `src/benchmark/strict\_v3.py` contains no scoring code : it normalises the contract per vulnerability and derives a conservative value for earlier ground truths. The `confirmed.*` metric is built in the comparator by prefixing the funnel stage names (`src/benchmark/funnel.py`) : `candidates`, `filtered`, `confirmed`. The `confirmed` stage denotes the deduplicated findings whose evidence has been independently validated against tool outputs. The code states it : a confirmation whose evidence is invalid is not a true positive. The stage is unavailable if the exploitation file or the provenance journal is missing ; it is never reconstructed from the survivors.

5.8 What these contracts do not prove

1. **Nothing about the ground truth.** The oracle is supplied by the operator ; each family records this in its scope. A false oracle makes all verdicts false.
2. **Nothing about admission.** Campaign admission requires the explicit adoption of the strict-v3 metrics, two independent families, three repetitions per posture and an attested resource parity : conditions not met by the calibrations.
3. **Nothing about arm parity.** SSH is not an identical task : on the LLM side, the model reads the result and judges equality with the control itself, while the deterministic arm compares the payloads ; the evaluator recomputes off-band. The budget-parity condition covers only declared budgets, not the hidden calls of the script arm.
4. **Nothing about a rare defect.** V0 is saturating : with three repetitions, it detects only a repeated defect.
5. **Nothing about the real space.** H2 holds for eight hand-written fixtures, a single variant schema and a single property.
6. **Nothing about semantics or injection** : these are H4 and H6, outside the scope of these contracts.
7. **Nothing about comparative cost** : the deterministic arm consumes no tokens.
8. **Certain guarantees belong to the written protocol**, not to a code assertion : the arm count of H2 and the invariant that an observation cannot answer two steps.

Where this lives.

— Contracts : `ssh\_reference.py`, `mqtt\_reference.py`, `api\_reference.py`, `v0\_control.py`, `mqtt\_formats.py` under `benchmarks/experiments/`.

6 Harnesses, and a sandbox that measures nothing

In brief. A harness is the instrument that puts two policies in front of the same situation : one contract file per family, a deterministic arm, a model arm, a controller for the laboratory and the evaluator. The same anatomy is repeated for SSH, MQTT, the API and the V0 control. In October the life cycle of these objects was unified into one construction function, so that a run missing part of the governance surface is refused at construction rather than misbehaving later.

The sandbox and its honest limit. A sandbox was added that runs the deterministic arms on any workstation and in continuous integration, without a broker, a model or a laboratory. It reproduces the H2 lesson offline : the declared parser fails where a script reading by convention succeeds. It *instruments* a contrast ; it measures nothing about a model, and its guard observes Python events, not system calls. The report therefore says `guard\_clean`, never "isolated".

6.1 Anatomy of a harness

A harness is a set of files with one responsibility each. For MQTT, which gives its complete form : the contract validates the briefing, binds the tool to the execution boundary, provides the deterministic arm and recounts the run through the evaluator. The model arm is a separate file, and its H2 variant receives no parser. The live controller builds the model, journals every transport call and writes the decision, the cost summary and the run metadata. The deployment uses the laboratory, submitting operations to the hypervisor.

The same anatomy is repeated for SSH, the API and the V0 control, with one difference : the H2 arms — the contract and its evaluator — need neither a deployment nor a broker, because their corpus is frozen. Its controller `mqtt\_formats\_live.py` does require the laboratory runner and the model catalogue, so it is not a sandbox instrument.

6.2 The shared core

The core exists for a precise reason, written in its header. The governance boundary reads a fixed set of attributes from the run object with defaults ; an omission therefore manifests only when a tool call misbehaves. Each harness used to rebuild that object by hand. The core replaces the copies with a single construction, which enumerates thirteen required attributes — the run directory, three budgets, the archival lock, the call counter, the start clock, the limit reason, the scope, the context, the policy and the cost tracking — and refuses an incomplete one.

One attribute deserves attention : the evidence-integrity flag. The limit check reads it first and raises if it is set, and the archiver sets it when the archive cannot be written. A harness that forgot it would pursue a run whose archive is holed, and the result would look complete. The refusal is two-level : at construction, and on any object through a replay check.

6.3 The sandbox

The sandbox proves, on a workstation as in continuous integration, that a harness executes without a broker, without a model and without a laboratory : it loads the frozen corpus of eight fixtures and, for each one, validates the fixture, opens a journal, runs the chosen arm, writes the artefacts and recounts through the existing evaluator. Five artefacts are produced per trial, and a report aggregates them with a guard block.

The arm is an explicit parameter and an unknown name raises an error. This was a correction : the first version read the arm from the corpus, which does not declare it, so the conventional arm was unreachable and the report invented the policy it believed it had tested.

6.4 The guard, and what it does not see

The guard relies on the audit hook of the interpreter. It stays installed but enforces only inside an explicit context, and it is rebound at each run so that a second run does not observe the first run's directory. It refuses sockets, subprocesses, the process-spawning family, native library loading, and writes outside the authorised directory.

Its honesty is itself a result worth keeping. A read-only audit broke the first version with two measured escapes — a spawned process and a native call through the C library — although the report announced a hermetic execution without violation. Three changes followed : the key now says `guard\_clean`, which states only that no listed event occurred ; a guard block declares the mechanism, the blocked events and the blind spots ; and the isolation claim is explicitly false. The last is false by construction, because an audit hook observes events of the interpreter, not system calls.

Table 8 – What the guard covers, and what it leaves unobserved.

Covered, with refusal raised	Unobserved
Sockets : connection, resolution, sending, listening	Native code reached without the loading library, subprocess or spawn
Processes : subprocess, system, exec, fork, spawn	Writes through a descriptor opened before arming
Native libraries : loading, symbol resolution, call	System calls issued outside the interpreter
Rename and delete operations	Any behaviour after disarming : the hook remains installed but inert
Writes outside the authorised directory, on resolved paths	No guarantee of process isolation

6.5 What the sandbox demonstrates, and what it does not

On the eight frozen fixtures, the declared-parser arm succeeds in six trials : the two failures are the raw variants, where the declared parser does not cover the schema and abstains, hence an erroneous conclusion. The arm reading by convention succeeds eight times out of eight. Success is not counted on the conclusion alone : the evaluator requires a correct conclusion, no contradiction and no erroneous verdict.

The lesson is the same as H2. The failure does not stem from a lack of intelligence but from a boundary of the declared parser, and a script that tries restores the coverage. The sandbox instruments that contrast ; it does not measure it on a model. No model arm appears there, no pre-flight attestation is produced, and no admissibility flag emerges from it.

Five limits must be read against those results. The sandbox involves no model, so its success is a property of the harness, not an experimental result, and the report carries `claims\_measurement: false`. The guard is a Python observation, not a proof of isolation. The negative tests prove the refusals, but only at unit level : no end-to-end test confirms that the installed hook is reached during a run. The corpus is frozen, with one scenario and eight fixtures, which suffices to instrument a contrast but not to estimate a rate. And the sandbox bypasses the governance boundary it accompanies, calling the arm directly and writing its own journal, so the core and the guard remain two instruments verified in isolation.

6.6 What the tests prove

Twenty-four tests cover the core and the sandbox. What they prove :

— an incomplete run is refused at construction ;

— the call budget is raised by the wrapper rather than by the check ;
— the eight fixtures produce the five artefacts and a consistent report ;
— the arm is declared rather than read from the corpus ;
— the guard refuses writes outside the directory, the network, subprocesses and three native-library events.

What they do not prove : no test executes a model arm, none measures an interval, and none makes a sandboxed run attempt a forbidden operation, so the sanction is proved at unit level rather than end to end.

Where this lives.

— Harnesses, the shared core and the sandbox live under `benchmarks/experiments/` ; tests in `tests/test\_harness\_core.py` and `tests/test\_sandbox.py`.
— The core is `build\_run` ; see also `REQUIRED\_ATTRIBUTES` and `require\_attributes`.

7 Execution infrastructure and delivery pipeline

In brief. An audit cannot be measured on a real network : the result would depend on whatever is exposed that day, and two runs would not be comparable. LANCE therefore audits networks it has itself built from a description, and delivers its platform through a push, validate, deploy chain to three instances, none of them on a workstation.

What this buys, and what it does not. It buys repeatability and attribution : a run that skipped the reset would inherit the previous trial's state, and its result would no longer be attributable to it. It does not buy availability : the single host, the single container and the single runner are declared points of failure, the relay to the university model is not a supervised service, and a failed deployment leaves the instance stopped with no automatic rollback.

7.1 Why a laboratory at all

An audit cannot be measured on a real network. The result would depend on whatever happens to be exposed that day, could not be repeated, and would not be comparable between two runs. LANCE therefore audits networks that it has itself built, from a description, and throws away afterwards.

A scenario is that description : which devices exist, which services they run, which weaknesses are installed, and which connections are allowed. Deploying it means materialising those devices as virtual machines on a hypervisor, so that the audit sees a real network rather than a simulation, and destroying them afterwards so that the next trial starts from the same state.

Proxmox provides that hypervisor. It is the host on which the scenario devices are created as LXC containers — lightweight, unprivileged Linux containers, each acting as one device of the scenario : a web service, an IoT service, a gateway, or any other role the scenario describes. The scenario never runs on the workstation and never touches a real network : it exists only on the Proxmox host, for the duration of a trial.

Ansible plays the other half. It configures each container from the scenario description — installs the service, sets the credentials, injects the weakness — and resets it to a clean state before the next trial. Deployment, configuration and reset are the three operations that make a trial repeatable ; the hypervisor alone would give none of them.

The cycle is therefore : deploy the scenario as containers, configure and inject, run the audit, destroy. A run that skipped the reset would inherit the previous trial's state, and its result would no longer be attributable.

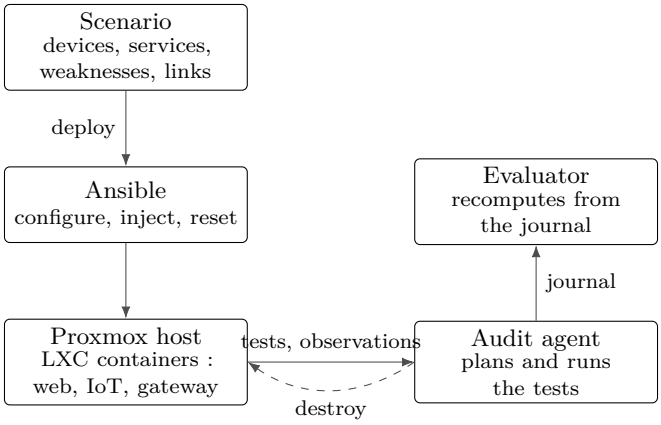


Figure 4 – The laboratory cycle. A scenario is deployed as virtual devices on the Proxmox host, configured by Ansible, audited, then destroyed. The evaluator reads the journal, never the devices.

The platform itself runs in one unprivileged container on that same Proxmox host. It carries the service, the runner and the artefacts. Two interfaces are attached : one on the management network, in DHCP, and one on the scenario network, with no gateway, which confines the scans to the isolated segment. An output firewall lets through only what the platform needs — name resolution, HTTP, SSH, the hypervisor API, the tunnel of the overlay network and the package cache — and rejects everything else.

One detail of documentation matters for reproducibility : the host name appears in the project's own guides but not in the versioned inventory, which is a template. The only constant readable in the repository is the container identifier, 200. The actual configuration lives outside the repository, and the report says so rather than pretending otherwise.

7.2 The three instances

Three instances share the container, one per branch, each with its own environment, database, configuration and artefacts : the stable instance on the main branch, and one working instance per development branch. The deployment script fixes the slot, the port, the directory and the service, and rejects any other name. Every instance answers a status route that announces its slot, its branch, its commit and whether the laboratory lock is active ; after start-up, the script checks that the announced commit is exactly the one that was validated.

The lock is a single file, taken in exclusive mode by the deployment script before it switches the checkout, and removed only by hand. The same value is passed to the three instances, which lets them share one physical laboratory. Waiting is cooperative, in memory and without arrival ordering, so it is an etiquette between instances, not a security isolation between users.

7.3 The runner and the delivery pipeline

The GitHub Actions runner is installed in `/opt/actions-runner`, registered with the label `nato-master` and run as a systemd service.

The pipeline reads as four links. A `push` to `main`, `dev/1` or `dev/2`, or a merge request into these branches, triggers `.github/workflows/benchmark-integrity.yml`. The first job runs `.github/scripts/ci\_scope.py`, which classifies the modified files and publishes the `full` flag. The second builds the image of the isolated worker and checks the absence of ground truth. The deployment jobs then call `.github/workflows/update-master.yml`, which runs the deployment script with the slot, the validated SHA, the branch and the remote repository.

Table 9 – Triggers and paths through the pipeline.

Event	Path	Deployment
documentation `push`	classification to `full=false` : diff of the previous commit and whitespace check	none
code `push`	full tests, scenario and Ansible checks, sandbox, worker image	`main` to 8501, `dev/*` to 8502 or 8503
Merge request	same path according to classification	none
Manual trigger	`full=true` forced	chosen slot, or checks only
Version tag	`full=true` forced	`main` only

The light path covers the Markdown, LaTeX, PDF and image files under `docs/`, `objectifs/` and `research/`, as well as a closed list of presentation files. The classification keeps deletions and the two paths of a rename ; any undetermined event falls back to the full path, and a failure blocks the deployment. A documentation push therefore restarts no instance.

On the machine side, the update script fetches the SHA and applies a fast-forward merge for `main`, or a detachment after checking for a clean tree for the development branches ; it then verifies that `HEAD` equals the validated SHA. Before that, the service is stopped, the lock acquired, and the instance queried in order to refuse mutating active or pending work. In the event of a failure after stopping, the service remains stopped : no automatic rollback to an earlier version is performed.

7.4 Corrections to the deployment script

The history of the deployment script is a short sequence of corrections made on 29 October 2026, the day the container was rebuilt and every automatic deployment stopped working. Four families of correction came out of it, and each answers a concrete failure rather than a matter of style.

First, the script assumed that it inherited the host's identity, the root user and the presence of the neighbouring instances. A rebuilt container breaks all three assumptions, so each became an explicit check. Second, the host identity is now read from the overlay network interface rather than from a constant, which removes the last value that a rebuild can invalidate. Third, privilege escalation loses the environment : the path must be absolute, the script is re-executed through a shell, and both the runner name and the deployment settings are carried explicitly. Fourth, a download failure used to stop the deployment and leave the instance stopped ; it is now tolerated and named.

The same identity logic now serves the benchmark controllers, and a laboratory lock variable allows an operator to suspend the check explicitly rather than by disabling code.

7.5 The UMONS model and its relay

The `ollama-umons` provider is reached through a local SSH transfer on the Proxmox host, towards the university cluster (`docs/guides/umons-llm.md`). The path was identified and restored on 4 October 2026, with a successful minimal generation ; the authentication observed is interactive. The restoration command is documented, as are the listening checks. The guide is explicit : neither start-up at boot, nor reconnection after the process ends ; the administration of the downstream GPU server remains to be confirmed with the administrator. The fragility is established in the facts : the relay went down again on 22 October, and it was the provider pre-flight that named the failure without executing any arm.

7.6 The outage of 29 October

The outages actually traced in the repository for that day are those of the container rebuild, which rendered obsolete the fixed address and the assumption of a privileged runner. By contrast, *no mention of a multipath default route on the laboratory router exists in the repository* : the search on the corresponding terms returns nothing. This fact, observed in session but not recorded, must not be presented as documented. It constitutes a gap in the project's memory, since the same outage will recur at the next router restart.

7.7 Known fragilities

— **Unreplicated failure points** : single Proxmox host, single container, single runner. The runner registration tolerating failure, an absent runner does not block the playbook but blocks every deployment.
— **UMONS relay** : no supervised service, interactive authentication, address still fixed in the examples of the guide although commit `e54da24` claimed to have stopped fixing addresses in the documentation. Restarting LANCE does not repair the relay.
— **Cooperative lock** : without arrival ordering, in memory, not covered for an Ansible script launched directly, an old checkout or a manual intervention. This is not security isolation between collaborators.
— **Manual recovery** : a failure after stopping leaves the service stopped, with no automatic rollback.
— **Host identity** : without a declared variable, the identity depends on the presence of any address on the Tailscale interface ; the script does not verify that this host is indeed the one that should host the instance.
— **Partial documentation** : the name of the Proxmox host and the physical segment correspond only imperfectly to the playbook ; the versioned inventory is a template.

Where this lives.

— Playbook : `benchmarks/ansible/playbooks/deploy\_master.yml` (Debian 13, 20 GB, 4 GB memory, 2 cores, nesting, on boot).
— Slots and ports : 8501 for `main`, 8502 for `dev/1`, 8503 for `dev/2` ; status route `GET/api/environment` ; lock at `/var/lib/lance/lab.lock`.
— Commits : `6b0a9ff`, `433e2a3`, `a568c68` to `dcd5903`, `150eb8a`. Script : `scripts/deployment/deploy-instance.sh`.

8 The benchmark, and the distance left to a result

In brief. The benchmark is a register of hypotheses, not a list of scenarios. Six hypotheses are planned (H1 to H6), each behind a validity gate V0 that asks whether the instrument can reproduce a known control before anything downstream is read. Measured to date : V0 passes on MQTT and saturates ; H2 was measured on 22 October and its prediction held, but the gain of the model was absorbed by a script reading by convention ; one family (C2) was rejected on paper, and one (the vendor-register family) was built and rejected in an hour on 30 October.

What this means for the question of the project. The corpus was built around what signature rules find : at most 2.1 % of its entries need interpretation, and 84.4 % are detectable by signature. A duel on this corpus therefore measures above all the quality of the rules, which is exactly what H2 observed. The conclusion is not that the model is inferior ; it is that no situation in which the difference could appear has been produced yet.

8.1 The register, and the gate that is not applied

Before every hypothesis the register places a fidelity control : if a task solvable without inference fails on one of the two arms, no downstream result is readable. The threshold is frozen at 0.9 with 24 steps, 3 repetitions and a saturating control. The calibration pass returns a valid control on both arms, three times out of three, and the register draws from it a *partially passed* gate : the instrument is readable for parity of execution and for a repeated defect, not for a rare one.

Two caveats must be stated. The control covers MQTT only : the SSH harness keeps a fixed credential dictionary and is therefore not an instance held apart. And the threshold is not applied by the gate : the gate function reads the four outcomes on "all trials succeeded", while the threshold is computed separately and conditions nothing. With three repetitions the two formulations coincide, but by numerical coincidence, not by application of the threshold. This is one of the four contradictions the conclusion confronts.

Table 10 – Status of the register on 2 November 2026.

Item	Status	What this means
V0	partially passed	valid control on MQTT ; threshold frozen but not applied by the gate ; SSH not covered
H1	conditional, not opened	the identity-chaining family does not exist in the coverage plan
H2	measured on 22 October	prediction held, gain absorbed by an arm by convention
H3	not implemented	the public-contract families are planned
H4	not opened	the vendor-register family was built and rejected on 30 October
H5	conditional, planned	budget counter-test to be written
H6	planned as a sub-test	graftable onto H2 or H3, without its own simulator
C2	rejected	two adversarial reviews, dead on paper, no run

8.2 The corpus is built around what rules find

The catalogue enumerates 29 scenarios ; the derived index counts 288 expected entries, of which 282 are classified as detectable by signature and 6 as belonging to logic, that is 2.1 %. Two counts coexist and do not measure the same thing : the *mechanical* reading places every category except two in signature, while the *interpretive* reading judges, category by category, whether a well-designed rule can conclude on its own, and finds 243 entries out of 288, that is 84.4 %. Both are legitimate and must never be cited one for the other.

Table 11 – Distribution of the 288 entries by category and interpretive reading.

Category	Count	Interpretive reading
misconfiguration	70	signature
data_exposure	54	signature
no_authentication	50	signature
default_credentials	41	signature
info_disclosure	16	signature
code_injection	13	mixed
insecure_update	12	mixed
weak_crypto	10	mixed
cve	9	signature
privilege_escalation	4	chaining
broken_access_control	4	logic
missing_header	3	signature
auth_bypass	2	logic
Total	288	243 signature, 35 mixed, 4 chaining, 6 logic

The logic stratum occupies only three scenarios, and only 55 entries are attached to a chaining of at least two hops (`chainage` two or three). A global duel on this corpus would therefore measure above all the quality of the rules, which is exactly what H2 observed. The two counts come from two artefacts. The mechanical one is the binary `type` field of `benchmarks/labels/derived\_index.json` : 282 entries by signature, 6 by logic. The interpretive one sums, category by category, the coverage audit of 12 October 2026 (`objectifs/asiaccs-2027/audit-couverture.md`) : 243 decidable by a well-designed rule, 35 mixed, 4 chaining, 6 logic. The word *signature* therefore means 282 entries mechanically and 243 interpretively.

8.3 H2 : a prediction that holds and demonstrates nothing

The corpus contains eight fixtures, two per group, for 25 messages in total. The measurement of 22 October 2026 used two arms and one repetition ; the reference run that followed used three arms per fixture — 24 arms per repetition, 72 for three repetitions.

On the raw variants, the model arm obtains 0.5 (one fixture won out of two), the declared parser 0.0, and the third arm, which reads by convention without any key from the corpus, covers 1.0. The prediction holds in the literal sense : the model does better than a declared parser that cannot read a schema it does not know. But its gain is absorbed by a convention that no one had to declare, and a convention is a rule.

8.4 Inference : repaired, and still idle

The inference tool carried a structural defect. It computed a two-sided p-value around zero, so every outcome returned the same figure : an almost constant series, a highly dispersed series of the same mean and a series entirely negative all produced a null p-value. A model performing strictly worse than the rules looked as certain as one performing better.

The defect was repaired on 30 October 2026. The module is `src/benchmark/inference.py` ; the test is now one-sided against the pre-registered threshold : an exact sign test up to seven families, a paired bootstrap beyond, both centred on the threshold rather than on zero. Below three families no p-value and no interval are emitted ; a decision requires five families, because five agreeing families are the first that can reach the usual threshold at all.

Repairing the computation did not make a conclusion possible. A second obstacle stands behind the first, and it is not a defect of the tool : the number of independent families available. The trial configuration includes commit, fingerprints and budgets, so two distinct families

form two distinct configurations ; each configuration contains one family, and the inference requires several. No figure can be produced from families that do not exist.

8.5 A family built, then rejected in one hour

On 30 October 2026 a family was built far enough to be measured, and rejected. The record matters more than the rejection : it is the first time the project's own protocol produced a decision instead of a plan.

What was built. Not the family : a probe. One fixture of three registers — one that commands something, one inert, one cache — with real neighbour reads, test writes, and every read attributed to a single write. The probe applies two arms : a convention arm reading the unit and the change, and an arm reading the neighbours. Neither an oracle, nor the closure test, nor the eight fixtures were built : the rejection came first.

What existed before the probe was the design, and it resisted six recorded reviews. Its addressing model was established against raw protocol frames rather than through a client object, and reproduced by three readers. The earlier drafts had been checked by reading ; the frames checked the offset that the readings had assumed.

The trap. One register commands something and changes when written ; one is inert and does not change ; the third is a cache — it changes when written but commands nothing, and propagates its value into its neighbours. The convention arm classes the cache as a projection and scores two out of three. The arm reading the neighbours scores three out of three and states why the cache is not a projection.

Why it was rejected. Reading the neighbours is a rule, not an interpretation. Whether one places that rule inside the competent automation or outside it, the family measures nothing. Inside, the automation succeeds and the closure test rejects the family. Outside, the family measures an omission of the automation rather than a limit of interpretation. There is no third reading.

The filter that would have caught it applies in minutes and costs nothing :

> If the rule that solves the case can be written in one line, a script will have it. Do not build the family.

What survives the rejection. Three findings remain, and they are what the next attempt inherits. First, absence can be deduced : a register that does not change when written, after a declared test, is attested as non-commanding without any observation labelled "absent" in the journal — labelling would have put the answer in the journal. Second, a journal must be attributable : no observation may belong to two causes, and the probe found three defects of that kind. Third, the protocol works : six reviews validated a design that one hour of observation rejected, and the observation cost less than a campaign.

What the rejection does not prove. It does not refute H4. It shows that this mechanism, on this corpus, is crossed by a generic rule. A different mechanism could survive, and any future family must demonstrate, before construction, that no generic rule crosses it.

8.6 Campaign admission

The admission module combines pair conditions and campaign conditions. The condition for explicit adoption of the confirmation metrics is a parameter that defaults to false and is never set to true by the summary nor by the command line. The justification is a contract argument : the driver harnesses do not emit these counters. This is accurate about the current state, but not a proof of unsatisfiability : the condition would become true again the day a harness publishes these counters. Today, admission is therefore refused because that condition is not satisfied.

A second point, verified during drafting, concerns budget parity. The condition requires that the first value exists and that the two are equal, so a missing budget is correctly refused. The

module does not, however, verify in its other conditions that a missing budget is distinguished from a non-comparable one.

8.7 What is publishable today

The publishable element is the method, not a result of superiority. The repository supports no claim of comparative performance. What holds is consistent and verifiable :

— a falsifiable register of hypotheses, with the contrast declared before the fixture, a private oracle and a measurement recomputed from the log ;
— a measurement lesson from H2 : a prediction can hold literally and demonstrate nothing, because an arm by convention absorbs the gain ;
— a clean negative result on C2, documented before any grey area ;
— a defensible negative result on the vendor-register family, obtained in one hour instead of a campaign ;
— an honest inventory of the holes : a reduced logic stratum, an unknown protocol absent, noise not encoded.

8.8 What is not demonstrated

Must not be written as established : the superiority of the model ; a significant gap, since no independent family exists to compute one ; generality, H2 covering one schema, one question and one model ; a cost gain, the exploratory pilot counting one pair without an interval ; the validity of the threshold, which the gate does not apply ; and the integrity of the corpus, the official healthy control not being in the catalogue.

The other active studies are recorded as qualifying the instrument rather than showing an advantage : the plan carries 44 records and several hundred conditional arms, of which the qualifications actually executed — SSH in October, MQTT a few days later — validate the instrument. The three-repetition pilot yields twelve admissible pairs and computes no interval. The formulation to keep is the one of the historical study on agent versus automation : a real descriptive result, no general superiority established.

References

Internal sources are cited by path, with the date of the state they describe. No external publication is claimed for a result that does not exist.

— LANCE project. *Architecture, protocols and current state, report as of 30 October 2026.* research/2026-10-30-architecture-protocoles-avancement/.
— Rejection of the pilot family `s_registres_constructeur`, 30 October 2026. research/2026-10-30-famille-pilote-registres-constructeur/.
— Hypothesis register and the V0 gate. research/2026-10-21-hypotheses-benchmark/.
— H2 measurement protocol and runs. research/2026-10-21-hypotheses-benchmark/h2-protocole.md.
— C2 rejection. research/2026-10-21-c2-couverture/.
— Derived corpus index (`benchmarks/labels/derived\_index.json`) and its generator, `src.benchmark.derived\_index`.
— V0 fidelity control manifest (`benchmarks/experiments/v0-control-manifest.json`).
— Reference material for the current behaviour : `docs/architecture/run-modes.md`, `docs/benchmark/derived-index.md`, `docs/guides/development-instances.md`.

9 Conclusion

9.1 What this report can say, and what it cannot

The project assembles three instruments that do not offer the same guarantees, and the main source of confusion is to treat them as interchangeable. Section 1 gave the state as a whole ; this conclusion states what follows from it.

Table 12 – What each instrument guarantees and does not guarantee.

Instrument	Guarantees	Does not guarantee
Execution boundary (`executor.py`)	Each tool call is logged, bounded and compliant with the scope	System-level isolation ; the guards are application-level
Sandbox (`sandbox.py`)	No listed Python event occurs ; the deterministic arms run outside the laboratory	Native isolation ; any model measurement
Calibration harness	Contract, separate oracle, evaluator that re-reads the log	Campaign admission ; exact parity of the arms

The first instrument makes the audit traceable. The second makes the harness portable. The third makes the comparison possible. None of them proves that a model does better than a script, which was the object of the programme.

9.2 The contradictions identified between sections

Comparing the parts brought out four discrepancies, each verified against the code before being retained here.

The threshold of the V0 check is not applied by the gate. The protocol framework asserts that the gate applies `X=0,9` ; the benchmark part verifies that the gate is computed on “all successes” and that the threshold check is independent. The two formulations coincide at three repetitions but diverge beyond that. The report retains the reading of the code.

The H2 corpus does not encode the described failure mode. The protocol states that, for a given record, abstention is the only correct answer ; reading the oracle shows that it expects an exposure on the first message and a non-exposure on the third. The by-convention arm moreover converges exactly on the oracle. The protocol therefore describes an intention that the corpus does not encode.

The blocking of inference is not the one commonly alleged. The admission condition is cited as the cause ; reading shows that it is satisfiable in principle, and that the real constraint is the absence of independent families. This shift matters : it changes the priority of the work.

The routing failure of 29 October is not recorded. It accounts for six hours of interruption and the rebuilding of the container, but no document in the repository describes it. This is a gap in the project’s memory, not a disagreement between sections.

9.3 A common pattern : absorbed gain

Three episodes follow the same shape. In the H2 family, the model’s gain over the declared parser is taken up by a script arm that reads by convention. In the sandbox, the same contrast

is reproduced offline, without a model. In the rejection of C2, the targeted flaw turns out to be expressible from public inputs, and therefore closeable by a competent automation.

The pattern can be stated thus : *what distinguishes the model from the script, in the constructed families, is a limitation of the declaration rather than a capability of the model.* This is not a demonstration of inferiority ; it is the absence of a situation in which the difference appears. Yet such a situation is precisely the object of the programme.

9.4 What the project can assert today

Four assertions are defensible, each with its limitation.

An audit produces verifiable traces : each tool call is archived, referenced and bounded, and a lost piece of evidence interrupts the run rather than leaving a silent gap. The limitation is that these are application-level guards.

A harness can be executed outside the laboratory, without a broker or a model, under a safeguard whose blind spots are declared. The limitation is that it measures nothing : it instruments.

A third script arm absorbs a gain attributed to the model on the most documented family. The limitation is that this holds for eight fixtures and one property.

The corpus is mostly detectable by signature, which makes a null difference expected by construction. The limitation is that this concerns the current corpus, not the targeted situations.

9.5 What the project cannot assert

No superiority of the model is established. No confidence interval can be computed. No generalisation is possible from the existing measurements. The inference module, which produced the appearance of a measurement it did not provide, was repaired on 30 October 2026 : it now refuses to emit a figure it cannot defend. What remains is the obstacle behind it — the families that would give it something to compute do not exist.

9.6 The three decisions that govern what follows

A first decision concerns the independent families, now that the module can no longer produce an indefensible figure : to build them, applying the filter that one hour of observation established, or to publish a bounded methodological result and declare it as such. The filter matters more than the count : a family crossed by a generic rule measures nothing, and five of them measure nothing either.

A second decision concerns the opening condition : to write a competent reference automation, or to explicitly renounce the hypotheses that depend on it. This decision makes several families that are currently undecidable legible.

A third decision concerns the comparable arm. A comparison of the two policies supposes that both receive the same observations, and four mechanisms currently favour the rules : the phase-four probes only they may emit, a supplement that rewrites the scan files, the scanner findings added to every aggregation, and a benchmark score reserved for the full pipeline. Either give the model an equivalent access, or state that the comparison measures verification rather than interpretation.

These decisions together determine whether an inference can one day exist.

None of these decisions is technical in the narrow sense. All of them concern what one accepts not to be able to say.

9.7 A legible conclusion

The system is built, instrumented and documented. Its chain of evidence is sound : a separate oracle, an evaluator that recounts, a log that does not forgive a lost piece of evidence, a safeguard that declares its blind spots. The defects identified in this report were found by reading what the code does, not what it claims, and that is the best evidence that the verification discipline works.

The scientific question, however, remains open. The programme asked when a model does better than a script with identical tools ; the available data do not answer it, and the current arrangement would not allow an answer even with more machine time. This observation is not a failure of the work accomplished : it is the result of having looked at things squarely.

The most useful continuation is not to add a family, but to make falsifiable what today claims to be so.